

5. 分治演算法

分治：「一刀均分左右，兩邊各自遞迴，返回合併之日，解答浮現之時。」

「架勢起得好，螳螂鬥勝老母雞。在分治的架構下，簡單的資料結構與算法往往也有好的複雜度。」

這一章介紹分治演算法 (Divide-and-Conquer algorithm, DaC)，又稱分而治之演算法，也有人稱為各個擊破法。分治是一種非常重要的演算法思維模式與策略，有很多重要的演算法都是根據分治的思維模式，例如快速排序法、合併排序法、快速傅立葉轉換 (FFT)、矩陣乘法、整數乘法以及在一些在計算幾何的知名演算法都是分治的策略。相較於它的重要性，分治出現在題目中的比例卻不是那麼高，主要原因有二：第一，很多分治算法的問題都有別的解法的版本，樹狀圖的分治也往往歸類到動態規劃；第二，一些重要的分治算法已經被納入在庫存函數中，例如排序，還有一些則因為太難而不適合出題。即使如此，分治還是很重要必須學習的思維模式，當我們碰到一個不曾見過的問題，分治往往是第一個思考的重點，因為分治的架構讓我們很容易找到比暴力或天真做法要好的方法。

5.1. 基本原理

從名稱 `divide-and-conquer` 就可以看出來，分治算法的精神就是將問題切割後再克服，事實上，分治的算法都可以分成三個主要步驟：

1. 切割。將問題切割成若干個子問題，通常是均勻地切成兩個。
2. 分別對子問題求解。
3. 合併。將子問題的解合併成原問題的解。

這裡所謂的子問題切割，並不是將問題分成不同的問題，而是「相同問題比較小的輸入資料」，也因為如此，第二步驟對子問題分別求解其實是透過遞迴呼叫來解子問題，也就是說，除了終端條件外，第二步驟什麼都不必做。這裡就看出分治算法的迷人之處，在思考一個問題的解的時候，你不需要去想解的步驟，只要去想「如何將子問題的解合併成整個問題的解」就可以了。如果我們要計算的問題是一個函數 f ，而輸入資料是 s 。以下是一個典型的分治算法：

演算法 $f(s)$ ：

如果 S 已經很小，達到遞迴終端條件，則直接計算後回傳 $f(S)$ ，結束；

將 S 切割成 S_1 與 S_2 ；

遞迴呼叫計算 $f(S_1)$ 與 $f(S_2)$ ；

將 $f(S_1)$ 與 $f(S_2)$ 合併成 $f(S)$ 後回傳；

結束；

我們用簡單的例子來做具體的說明。

P-5-1. 最大值與最小值

在一個長度為 n 的整數序列中找出最大值與最小值。

(本題只為了說明，太簡單所以沒附測資)

這一題太簡單了，我們只是要用他來說明分治算法的精神。直接的做法：從頭到尾掃描一遍，記住目前看到的最大值，每走一步就嘗試更新最大值，這樣可以找出最大值；相同的方法再做一次找最小值。時間複雜度 $O(n)$ ，也不可能更好了，因為每個資料一定要被檢查過才可能確保答案的正確性。

我們來看看分治的想法。假設我們要計算陣列在區間 $[left, right-1]$ 中的最大與最小值，我們從中點把區間分成左右兩個區間 $[left, m-1]$ 與 $[m, right-1]$ ，對兩邊分別遞迴求解；左邊的最小與右邊的最小取比較小的就是全部的最小；相同的，左邊的最大與右邊的最大取比較大的就是全部的最大。遞迴的終結條件呢？如果區間裡面只剩一個數字，最小最大都是它。以下是這樣的想法實作出來的程式，為了讓大家多習慣結構，我們故意在這題用了 `struct`。副程式 `f()` 是一個遞迴函式，其作用在找出 `A` 陣列在 $[s, t)$ 區間的最大與最小值，請注意我們用了 C++ 習慣的左閉右開的區間，程式中先檢查遞迴終端條件，這裡設的是區間剩下一個元素，接著計算中點 m ，然後對左右兩邊分別遞迴呼叫求解，最後合併解回傳。

```
#include <bits/stdc++.h>
using namespace std;
struct P {
    int pmin, pmax;
};
// range [s, t)
P f(int A[], int s, int t) {
    if (s+1==t) return {A[s], A[s]};
    int m=(s+t)/2;
```

```
P le=f(A,s,m), ri=f(A,m,t);
return {min(le.pmin, ri.pmin), max(le.pmax, ri.pmax)};
}

int main() {
    int a[10]={3, 0, 7, -3, -10, 5, 1, 7, 6, -2};
    P x=f(a,0,10);
    printf("%d\n%d\n",x.pmin, x.pmax);
    return 0;
}
```

再次強調分治的重點只在如何合併解答，其它步驟都是非常簡單的，也因為只需要思考合併，分治的正確性往往是比較容易證明的。還有一個好處是複雜度往往很容易計算（當然是指一般情形下），在進行下個例子前，我們先來說明分治演算法的複雜度如何計算。

分治的複雜度

分治是一個遞迴的演算法，不像迴圈的複雜度可以用加總的方法或是乘法計算，遞迴的複雜度是由遞迴關係式 (recurrence relation) 所表達。計算複雜度必須解遞迴關係式。遞迴關係又稱為差分方程式 (difference equation)，解遞迴關係是個複雜的事情，超過本教材要講的範圍（也超過中學生的國際資訊奧林匹亞競賽的範圍），所以在這裡我們只說明一些常見的狀況。

分治演算法的常見形式是將一個大小為 n 的問題切成 a 個大小為 b 的子問題（相同問題較小的輸入資料），此外必須做一些分割與合併的工作。假設大小為 n 的問題的複雜度是 $T(n)$ ，而分割合併需要的時間是 $f(n)$ ，我們可以得到以下遞迴關係：

$T(n) = a \times T(n/b) + f(n)$ ，這裡我們省略不整除的處理，因為只計算 big-O，通常不會造成影響。這個遞迴式子幾乎是絕大部分分治時間複雜度的共同形式，所以也有人稱為 divide-and-conquer recurrences，對於絕大部分會碰到的狀況 ($a, b, f(n)$)，這個遞迴式都有公式解，有興趣的請上網查詢 Master theorem。以下我們只說明最常見的幾個狀況的結果，這些遞迴關係的終端條件都是 $T(c)=O(1)$ ，對某常數 c ，意思是當問題的輸入小到某個程度時可以常數時間求解。

遞迴式	時間複雜度	說明
$T(n) = T(n/b) + O(1)$	$O(\log(n))$	切兩塊其中一塊不需要，如二分搜
$T(n) = T(n/b) + O(n)$	$O(n)$	每次資料減少一定比例
$T(n) = 2T(n/2) + O(1)$	$O(n)$	例如找最大值
$T(n) = 2T(n/2) + O(n)$	$O(n\log(n))$	如 merge sort

$T(n) = 2T(n/2) + O(n \log(n))$	$O(n \log^2(n))$	分割合併花 $O(n \log(n))$
$T(n) = 2T(n/2) + O(n^2)$	$O(n^2)$	$f(n)$ 大於 n 的一次方以上，結果皆為 $f(n)$

上面描述的狀況都是子問題的大小減少一定比例，它的複雜度通常都是不錯的結果，某些分治的子問題的只減少常數的大小，這時候的複雜度通常都是差的。例如

$T(n) = T(n-1) + O(n)$ ，結果是 $T(n) = O(n^2)$ ，例如插入排序法。

架勢起得好，螳螂鬥勝老母雞

我們再來看一個例子，分治或許不會讓我們一下子找到效率最好的解，但是如果使用分治，即使合併的方法很天真，往往也很容易找到突破天真算法複雜度的解法。

P-5-2. 最大連續子陣列 (分治) (同 P-4-13)

有一個整數陣列 $A[0:n-1]$ ，請計算 A 的連續子陣列的最大可能總和，空陣列的和以 0 計算。

(檔案中的測資與 P-4-13 相同)

這一題我們在第四章的 P-4-13 看過了，當時我們先說明了天真的 $O(n^2)$ 算法以及超天真的 $O(n^3)$ 算法，最後給了一個 $O(n)$ 的算法，而且這個問題跟 P-4-12 (一次買賣) 是等價的問題。現在假設我們不知道 $O(n)$ 解法，重新以分治的思維來尋找突破 $O(n^2)$ 的方法。

如果我們要計算陣列在 $[L, R-1]$ 區間的最大連續和，首先將區間平均切成兩段 $[L, M-1]$ 與 $[M, R-1]$ ，其中 $M = (L+R) / 2$ 。要找的解 (子陣列) 可能在左邊、右邊、或者跨過兩邊，所以只要三者取最大就是最後的答案。左右可以分別遞迴求解，唯一要做的是就是找出跨過兩邊的最大和，對於左邊，我們就笨笨的從中點往左一直累加，計算每一個可能左端的區間和 (也就是左邊的 suffix-sum)，然後取最大；同樣的，對於右邊計算所有 prefix-sum 的最大，然後兩者相加就是跨過中點的最大區間和。以下是範例程式，除了分治的概念外，非常的直覺。

```
//P-5-2 max subarray, DaC, O(nlogn)
#include <bits/stdc++.h>
#define N 200020
using namespace std;
typedef long long LL;

// max subarray in [le, ri)
LL subarr(LL a[], int le, int ri) {
```

```

if (le >= ri) return 0;
if (le+1 == ri) return max(a[le], (LL)0);
int mid=(le+ri)/2;
// recursively solve left and right parts
LL largest=max(subarr(a, le, mid), subarr(a, mid, ri));
// find largest sum cross middle
LL lmax=0, rmax=0;
// max suffix sum of the left
for (LL sum=0, i=mid-1; i>=le; i--) {
    sum += a[i];
    lmax=max(lmax, sum);
}
// max prefix sum of the right
for (LL sum=0, i=mid; i<ri; i++) {
    sum += a[i];
    rmax=max(rmax, sum);
}
return max(largest, lmax+rmax);
}

int main() {
    LL n, a[N];
    scanf("%lld", &n);
    for (int i=0; i<n; i++)
        scanf("%lld", &a[i]);
    printf("%lld\n", subarr(a, 0, n));
    return 0;
}

```

事實上除了分治的概念外，找前綴和與後綴和的做法跟原來的 $O(n^2)$ 做法類似，只是原來的做法外面套了一個迴圈（嘗試所有左端），這裡則是外面套了一個分治的遞迴，這麼直覺的做法看起來複雜度要糟糕了，讓我們來算一下複雜度。若 $T(n)$ 是此算法在資料量為 n 時的複雜度，因為分割與合併的處理時間是 $O(n)$ ，所以 $T(n) = 2T(n/2) + O(n)$ ，答案是 $O(n \log(n))$ ！俗話說：架勢起得好，螳螂鬥勝老母雞。分治用的好，天真的合併做法也可以得到不錯的解。如果你去仔細思考其中的原因，你會發現，雖然每一次合併都是笨笨的做，但是任一個資料參與合併的次數只有 $\log(n)$ 次，所以所有資料參與到的總次數只有 $n \log(n)$ 。分治很迷人，雖然這一題有更好的算法。

在結束這一題之前，或許有人想問：這一題用分治是否也可以做到 $O(n)$ 呢？事實上不難，提高算法效率的原則，就是去檢查看看算法中有那些重複的計算，我們留給有興趣的人自己去思考，所附檔案中有一支 $O(n)$ 的範例程式。

5.2. 例題與習題

口渴的烏鴉

看下一個例題前先講一個笑話：有一隻口渴的烏鴉，找到了一個裝有一半水的水瓶，但是瓶口太小喝不到水，於是他飛呀飛的，去到潺潺流水的河邊，找到一些小石頭叼回來放進瓶子裡，烏鴉飛了幾遍，隨著小石頭放入瓶中，瓶中的水位終於慢慢升起，烏鴉很開心的喝水，他欣賞自己聰明的頭腦可以解決這麼複雜的問題。忽然間，他想到他剛才不是去了河邊好幾次嗎？

看下一個例題。

P-5-3. 合併排列法

有一個整數陣列 $A[0:n-1]$ ，請將 A 中的數字從小到大排列。

(本題為示範方法，無測資)

合併排序法是一個利用分治的經典排序法，放這個例題不是要解題，只是讓大家了解他是如何運作，下面是範例程式。函數 `merge_sort()` 負責將陣列的 $[le, ri-1]$ 區間從小排到大，第一步先檢查遞迴終端條件：剩下不到兩個就不必排了。否則計算中間點 m ，遞迴呼叫兩邊；然後重頭戲就是將兩邊已經各自排好序的序列合併成一個，我們啟用一個臨時的空間 `temp[]` 放置合併後的序列，用 j 指著右邊已經處理到的位置，用 k 指著要放入 `temp[]` 的位置。對於每個左邊的元素 $a[i]$ ：先把右邊比 $a[i]$ 小的 $a[j]$ 複製到 `temp[]`，然後再把 $a[i]$ 複製到 `temp[]`。 i 迴圈跑完後，右邊可能有剩下的，但是它們都已經在正確的位置上了，所以不需要處理。最後我們將 `temp[]` 中的內容拷貝回陣列原來位置。

```
// p-5-3 merge_sort
#include <bits/stdc++.h>
using namespace std;

// [le, ri)
void merge_sort(int a[], int le, int ri) {
    if (le+1 >= ri) return; // terminal case
    int m = (le+ri)/2;
    merge_sort(a, le, m); // recursively sort left part
    merge_sort(a, m, ri); // recursively sort right part
    // merge the two sorted lists
    int temp[ri-le], j=m, k=0;
    for (int i=le; i<m; i++) {
        while (j<ri && a[j]<a[i])
            temp[k++] = a[j++];
        temp[k++] = a[i];
    }
    // the remaining of right part are already on position
    // copy back to a
    for (k=le; k<j; k++)
        a[k] = temp[k-le];
}
```

```

    return;
}

int main() {
    int a[10]={3, 0, 7, -3, -10, 5, 1, 7, 6, -2};
    merge_sort(a, 0, 10);
    for (int i=0; i<10; i++)
        printf("%d ",a[i]);
    printf("\n");
    return 0;
}

```

合併排序法的時間複雜度是 $O(n\log(n))$ ，因為合併的部分需要的是 $O(n)$ ，所以複雜度遞迴式為： $T(n) = 2T(n/2) + O(n)$ 。

下一題是分治的經典題。

P-5-4. 反序數量 (APCS201806)

考慮一個數列 $A[1:n]$ 。如果 A 中兩個數字 $A[i]$ 和 $A[j]$ 滿足 $i < j$ 且 $A[i] > A[j]$ ，也就是在前面的比較大，則我們說 $(A[i], A[j])$ 是一個反序對 (inversion)。定義 $W(A)$ 為數列 A 中反序對數量。例如，在數列 $A = (3, 1, 9, 8, 9, 2)$ 中，一共有 $(3, 1)$ 、 $(3, 2)$ 、 $(9, 8)$ 、 $(9, 2)$ 、 $(8, 2)$ 、 $(9, 2)$ 一共 6 個反序對，所以 $W(A) = 6$ 。請注意到序列中有兩個 9 都在 2 之前，因此有兩個 $(9, 2)$ 反序對，也就是說，不同位置的反序對都要計算，不管兩對的內容是否一樣。請撰寫一個程式，計算一個數列 A 的反序數量 $W(A)$ 。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，代表數列長度，第二行有 n 個非負整數，是依序數列內容，數字間以空白隔開。 n 不超過 $1e5$ 數列內容不超過 $1e6$ 。

輸出：輸出反序對數量。

範例輸入：

```

6
3 1 9 8 9 2

```

範例結果：

```

6

```


給定一個數列 A ，計算 $w(A)$ 最簡單的方法是對所有 $1 \leq i < j \leq n$ ，檢查數對 $(A[i], A[j])$ ，但是時間複雜度 $O(n^2)$ 顯然不夠好。以分治思維來考慮。要計算一個區間的反序數量，將區間平分為兩段，一個反序對的兩個數可能都在左邊或都在右邊，否則就是跨在左右兩邊。都在同一邊的可以遞迴解，我們只要會計算跨左右兩邊的就行了，也就是對每一個左邊的元素 x ，要算出右邊比 x 小的有幾個。假設對左邊的每一個，去檢查右邊的每一個，那會花 $O(n^2)$ ，不行，我們聚焦的目標是在不到 $O(n^2)$ 的時間內完成跨左右的反序數量計算。記得第二章學過的排序應用，假設我們將右邊排序，花 $O(n \log(n))$ ，然後，每個左邊的 x 可以用二分搜就可算出右邊有多少個小於 x ，這樣只要花 $O(n \log(n))$ 的時間就可以完成合併，那麼，根據分治複雜度 $T(n) = 2T(n/2) + O(n \log(n))$ ， $T(n)$ 的結果是 $O(n \log^2(n))$ ，成功了！

以下是這樣寫出來的程式，其中第 4~13 行是自己寫二分搜計算一個陣列中比某數小的元素個數，如果喜歡使用 STL 的可以用第 26 行的函數呼叫就可以了。計算反序數的遞迴函數在第 16~29 行，第 18 行檢查終端條件(剩下不超過一個元素的反序數自然是 0)，第 20 行遞迴呼叫左右之後，在第 21 行將右邊排序，然後用一個迴圈對每一個左邊的元素計算右邊有幾個比他小。

```

00 // p_5_4_a nlog^2(n) method
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 // binary search the num <x from a[0] to a[len-1]
05 int bsearch(int a[], int len, int x) {
06     if (a[0] >= x) return 0;
07     int pos=0;
08     for (int jump=len/2; jump>0; jump>>=1) {
09         while (pos+jump<len && a[pos+jump]<x)
10             pos += jump;
11     }
12     return pos+1;
13 }
14
15 // interval= [le,ri), return #inversion
16 LL inv(int ar[], int le, int ri) {
17     int i,j,mid = (ri+le)/2; // middle point
18     if (le+1 >= ri) return 0; // terminal case
19     // recursively solve left and right parts
20     LL w = inv(ar, le, mid) + inv(ar, mid, ri), cross=0;
21     sort(ar+mid, ar+ri); // sort the right part
22     // for each in left, binary search to find #inversion
23     for (i=le; i<mid; i++) {
24         cross += bsearch(ar+mid, ri-mid, ar[i]);
25         // following is using STL binary search
26         // cross += lower_bound(ar+mid, ar+ri, ar[i]) - (ar+mid);
27     }
28     return w + cross;
29 }
30 int main() {

```



```

31     int i, n, ar[100010];
32     scanf("%d", &n);
33     for (i=0; i<n; i++) scanf("%d", ar+i);
34     printf("%lld\n", inv(ar, 0, n));
35     return 0;
36 }
37

```

程式並不難，複雜度也算不錯了，但可不可以精益求精做到 $O(n\log(n))$ 呢？剛才只是單邊排序，所以很多次的二分搜花了 $O(n\log(n))$ 。假設兩邊遞迴呼叫時，回傳都是排好序的，我們就可以不必寫二分搜了，因為連續二分搜不如一路爬過去。所以副程式可以改成下面這樣。

```

// return the inversion in [le, ri) ans sort them
LL inv(int a[], int le, int ri) {
    if (le+1 >= ri) return 0; // terminal case
    int m = (le+ri)/2;
    // recursively solve the left and the right parts
    LL w=inv(a, le, m) + inv(a, m, ri), cross=0;
    // count a[j]<a[i] for i<m<=j
    int j=m;
    for (int i=le; i<m; i++) {
        while (j<ri && a[j]<a[i]) // until >=a[i]
            j++;
        cross += j - m; // num <a[i]
    }
    sort(a+le, a+ri); // sort before return
    return w + cross;
}

```

雖然避掉了二分搜，但是要排序，所以合併階段還是花了 $O(n\log(n))$ ，整個程式的複雜度還是 $O(n\log^2(n))$ 並沒有降下來。

再看看這個程式是不是有點熟悉呢？計算右邊有幾個比 $a[i]$ 小的部分不就幾乎是在合併左右兩個排好序的陣列嗎！確實，這支程式跟 P-5-3 的合併排序法少有 87% 以上相同，有沒有突然想起前面講的那隻口渴的烏鴉？是的，我們剛才在一個很像合併排序法的程式裡面呼叫 `sort()`！就像是烏鴉只想著要找石頭而忘了最終目的是喝水。所以，其實呼叫排序根本是不必要的，我們在合併兩個排好序的序列時，本來就幾乎已經算了反序數了。以下是這樣的寫法，我們只要把合併排序法稍加修改就可以了，時間複雜度也跟 merge sort 一樣是 $O(n\log(n))$ 。

```

// p-5-4 inversion number DaC  $O(n\log n)$ 

```

```

#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

// return the inversion in [le, ri) and sort the array
LL inv(int a[], int le, int ri) {
    if (le+1 >= ri) return 0; // terminal case
    int m = (le+ri)/2;
    // recursively solve the left and the right parts
    LL w=inv(a, le, m) + inv(a, m, ri), cross=0;
    // merge the two sorted lists
    int temp[ri-le], j=m, k=0;
    for (int i=le; i<m; i++) {
        while (j<ri && a[j]<a[i])
            temp[k++] = a[j++];
        temp[k++] = a[i];
        cross += j - m; // num <a[i]
    }
    // the remaining of right part are already on position
    // copy back to a
    for (k=le; k<j; k++)
        a[k] = temp[k-le];
    return w + cross;
}

int main() {
    int i, n, ar[100010];
    scanf("%d", &n);
    for (i=0; i<n; i++) scanf("%d", ar+i);
    printf("%lld\n", inv(ar, 0, n));
    return 0;
}

```

下一題與第四章的 P-4-15 是一樣的，放在這裡當作習題是希望有興趣的人可以練習分治的寫法，後面會給予提示。

Q-5-5. Closest pair (同 P-4-15, 分治版) (@@)

平面兩點的 L_1 距離為兩點的 x 差值與 y 差值的和，也就說，如果兩點座標是 (a,b) 與 (c,d) ，則 L_1 距離是 $|a-c|+|b-d|$ 。輸入 n 個點的座標，請計算出 L_1 距離最近兩點的 L_1 距離。

Time limit: 1 秒

輸入格式：第一行為一個正整數 n ，接下來 n 行，每行兩個整數 x 與 y 代表一點的座標。 n 不超過 $1e5$ ，座標值絕對值不超過 $1e8$ 。

輸出：最近兩點的 L_1 距離。

範例輸入：

```
4
-1 5
4 0
3 1
-2 -3
```

範例結果：

```
2
```

Q-5-5 提示：將點依照 x 座標排序後，將所有點均分為左右兩部分，以遞迴分別求出左右的最近距離在取最小值，在合併階段的工作是找出來左右各一點的最小距離。假設目前已找到的最小距離是 d ， x 為中值的點座標是 (p, q) ，那麼兩邊的點只有 x 座標在 $[p-d, p+d]$ 區間的點需要計算。如果兩邊的點已經依照 y 座標排序，對左邊的任一點 (x, y) ，右邊的點中只有 y 座標落在 $[y-d, y+d]$ 區間的點需要計算與 (x, y) 的距離。在 P-4-15 掃描線算法時，我們必須使用一個動態的資料結構在點逐步加入時來維持 y 座標排序，但是使用分治時，利用 merge sort 類似的方法，只需要用簡單的陣列就可以做到了，時間複雜度一樣是 $O(n \log(n))$ 。

下一題也是見過的，之前出現在第三章的習題，這裡當作例題說明分治的寫法。

P-5-6. 線性函數 (同 Q-3-14, 分治版)

有 N 線性函數 $f_i(x) = a_i x + b_i, 1 \leq i \leq N$ 。定義 $F(x) = \max_i f_i(x)$ 。輸入 $c[i], 1 \leq i \leq m$ ，請計算 $\sum_{i=1}^m F(c[i])$ 。

Time limit: 1 秒

輸入格式：第一行是 N 與 m 。接下來有 N 行，依序每行兩個整數 a_i 與 b_i ，最後一行有 m 個整數 $c[1], c[2], \dots, c[m]$ 。每一行的相鄰數字間以空白隔開。 $N \leq 1e5$ ， $m \leq 5e4$ ，輸入整數絕對值不超過 $1e8$ ，答案絕對值不超過 $1e15$ 。

輸出：計算結果。

範例輸入：

```
4 5
-1 0
1 0
-2 -3
2 -3
4 -5 -1 0 2
```

範例結果：

每一個線性函數對應到平面上一條直線，這一題需要一點幾何特性，簡單的說，越往右邊，斜率大的函數值(Y 值)會越大。因此，假設我們先將直線函數依照斜率由小到大排序，對於某一點 $x = c$ 時，先找出在此點的最大函數值是某個 f_i 。那麼，在 $x > c$ 時，我們不需要計算斜率比 f_i 小的函數值；同時，在 $x < c$ ，不需要斜率比 f_i 大的函數值。為了分治的效率，我們每次抓 x 的中值，計算出哪一條直線在此的函數值最大(笨笨的算就可以)，然後遞迴左右，以下是範例程式。

為了排序，我們定義了一個結構來存每一條直線，也定義了一個比較函數：先比斜率，斜率相同比截距，當然也可以用 STL 的 `pair` 來寫而略過比較函數。程式重頭戲就是遞迴函數 `dc(l1, r1, l2, r2)`，這個遞迴函數計算點 `point[l2]~point[r2-1]` 在直線 `line[l1]~line[r1-1]` 的最大值總和，做法依照上一段的說明，其實很簡單。

```
// P-5-6. Let F be the maximum of a set of linear function.
// Find F(c[1]) + F(c[2]) + ... F(c[m]), divide and conquer
#include <bits/stdc++.h>
using namespace std;
#define N 100010
using namespace std;
typedef long long LL;
struct Line {
    int a, b; // y=ax+b
} line[N];
// compare by slope first
bool cmp(Line p, Line q) {
    if (p.a == q.a)
        return p.b < q.b;
    return p.a < q.a;
}
int point[N];

// for line [l1,r1), point [l2,r2)
LL dc(int l1, int r1, int l2, int r2) {
    if (l2 >= r2) return 0; // no points
    int mid = (l2+r2)>>1;
    // find max y of the middle point
    LL max_y = LLONG_MIN, who;
    for (int i=l1; i<r1; i++) {
        LL y=(LL)point[mid]*line[i].a+(LL)line[i].b;
        if (max_y < y) {
            max_y = y;
            who = i;
        }
    }
    // left side need only smaller slope, right side larger slope
    return max_y + dc(l1, who+1, l2, mid) + dc(who, r1, mid+1, r2);
}
```

```

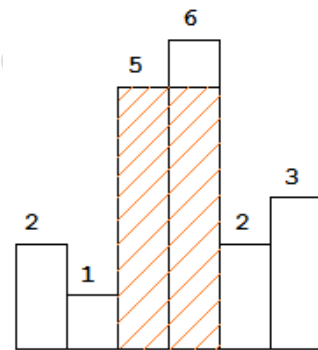
int main() {
    int i,m,n;
    scanf("%d%d",&n,&m);
    for (i=0;i<n;i++) {
        scanf("%d%d",&line[i].a,&line[i].b);
    }
    for (i=0;i<m;i++) scanf("%d",point+i);
    sort(line, line+n, cmp);
    sort(point, point+m);
    printf("%lld\n",dc(0,n,0,m));
    return 0;
}

```

下一題是網路上有很多解法的例題。

P-5-7. 大樓外牆廣告 (分治版)

有一條長街林立著許多大樓，每棟樓高高低低不一定，但寬度都是相同。現在想在牆面上找一塊面積最大的矩形來作外牆廣告，此矩形的一對邊必須平行地面，假設每一棟樓的寬度都是 1 單位。以右圖為例，有六棟樓，高度依序為 (2, 1, 5, 6, 2, 3)，最大矩形如圖中標示的部分，面積為 10。



Time limit: 1 秒

輸入格式：第一行 n ，代表有 n 棟樓，第二行有 n 個非負整數，依序代表從左到右每棟樓的高度。 n 不超過 $1e5$ ，樓高不超過 $1e8$ 。

輸出：最大矩形的面積。

範例輸入：

```

6
2 1 5 6 2 3

```

範例結果：

```

10

```

天真的解法是嘗試所有左右端區間 $[i, j]$ ，對於每個區間最大可能的高度是在此區間的最小樓高，如果對每一個區間都跑一個迴圈找最小高度，需要花 $O(n^3)$ ，那也天真的太過分了。類似 maximum subarray，我們可以對任一個固定左端 i ，一個迴圈跑完所

有可能的右端，過程中不斷的更新 prefix-minimum， $O(n^2)$ 就行了。這個解法太簡單了，所以也不需要放上版面來。我們來想想看分治的思維。

分治的想法當然是每次要算某個區間的解，問題是如何分割以及如何合併。假設第 i 棟大樓的高度 $h[i]$ 是全部高度中最低的，那麼，能跨 i 左右的最大矩形，他的水平位置是整個區間，高度就是 $h[i]$ 。既然其他的矩形都不可能跨過 i 的左右，我們就可以將區間切割成左右兩塊，分別遞迴求解，在所有找到的解中最大的就是最後的解。以下這樣想法之下的範例程式。

```
// p_5_7 divide and conquer at smallest,  $O(n^2)$ 
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
#define N 100010
LL h[N];
// largest rectangle in range [s, t)
LL rect(int s, int t) {
    if (s >= t) return 0;
    if (s + 1 == t) return h[s];
    // find min height
    int m = min_element(h + s, h + t) - h;
    LL max_rect = (t - s) * h[m];
    // recursively solve the left and right
    max_rect = max(max_rect, rect(s, m));
    max_rect = max(max_rect, rect(m + 1, t));
    return max_rect;
}

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%lld", &h[i]);
    printf("%lld\n", rect(0, n));
    return 0;
}
```

上面這個分治的時間複雜度是多少？必須看切割的狀況，複雜度遞迴式是：

$$T(n) = T(i) + T(n-i-1) + O(n),$$

如果運氣很好，每次切割左右都不小於一定比例，總時間複雜度會是 $O(n \log(n))$ ，但很不幸的，如果每次的最低點都在邊緣，例如說原輸入高度是遞增或遞減的，時間複雜度會高到 $O(n^2)$ 。能改善嗎？如果每次找最小值都可以很快的查詢出來，那時間複雜度是可以改善的，但必須動用到比較複雜的資料結構來做區間最小值查詢 (RMQ, range minimum query)。

假使我們從一定要降低複雜度的需求來思考。前面見過：架勢起的好，螳螂鬥勝老母雞。分治要有好的效率，分割時最好是平均分割，切成兩塊一樣大的，如此一來，只要合併花的時間小於 $O(n^2)$ ，整體的複雜度就會降下來了。均勻切割成兩塊不是問題，中點一切就行，問題在合併，也就是找出跨中點的最大矩形，我們來試試看。

假設目前要處理的區間範圍是 $[s, t-1]$ ，中點是 m ，那麼跨過中點矩形的高度顯然不能超過 $h[m]$ 。在高度不小於 $h[m]$ 的條件下盡量延伸左右範圍直到不能延伸為止，也就是會找到一個區間 $[i+1, j-1]$ ， $h[i] < h[m]$ 或者已達左邊界，而且 $h[j] < h[m]$ 或者已達右邊界。這樣我們可以求得以高度為 $h[m]$ 的最大矩形。每一次我們嘗試逐步下降高度，然後再往外擴增範圍，因為要逐步下降，所以下一次的高度基本上是 $\max(h[i], h[j])$ ，除非其中之一以超越邊界，那就取另一端。重複以上的步驟，我們會嘗試過區間內所有可能的最高高度，找到最大的矩形。程式碼其實不難，但要小心邊界。

在擴增範圍的過程中， i 只會由中點往左邊跑，而 j 只會從中點往右邊跑，所以合併的步驟只會花線性時間，因此整體的複雜度是 $O(n \log(n))$ 。因此，利用分治的架構，我們寫了一個不複雜的合併步驟，就達到不錯的時間複雜度。

```
// p_5_7 divide and conquer at middle,  $O(n \log n)$ 
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
#define N 100010
LL h[N];
// largest rectangle in range [s, t)
LL rect(int s, int t) {
    if (s >= t) return 0;
    if (s + 1 == t) return h[s];
    int m = (s + t) / 2;
    // recursively solve the left and right
    LL max_rect = max(rect(s, m), rect(m + 1, t));
    // merge step, find rectangle cross middle
    LL i = m, j = m, height = h[m], largest = 0;
    // each time, find maximal [i+1, j-1] of given height
    while (i >= s || j < t) {
        // choosing smaller as the next height
        if (i < s) height = h[j];
        else if (j >= t) height = h[i];
        else height = max(h[i], h[j]);
        // extend the boundary
        while (i >= s && h[i] >= height) // left boundary
            i--;
        while (j < t && h[j] >= height) // right boundary
            j++;
        largest = max(largest, (j - i - 1) * height);
    }
    return max(max_rect, largest);
}
```



```

int main() {
    int n;
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%lld", &h[i]);
    printf("%lld\n", rect(0, n));
    return 0;
}

```

P-5-7 存在更好的解法。分治程式的合併階段考慮的是左右邊各一個區段，如果你把他想成左邊的一個區段與右邊的一點如何合併解答，就會是掃描線演算法，範例程式中有這樣的解法，但我們這裡只要是說明分治，所以就留給有興趣的人自己去思考一下。

下一個習題是在第三章 Q-3-12 出現過的完美彩帶，當時使用的技巧是滑動視窗，這一題也可以使用分治法，合併階段要找跨過中點的解，只要能在 $O(n)$ 完成合併階段，整體複雜度就可以做到 $O(n \log(n))$ 。

Q-5-8. 完美彩帶 (同 Q-3-12，分治版) (APCS201906)

有一條細長的彩帶，總共有 m 種不同的顏色，彩帶區分成 n 格，每一格的長度都是 1，每一格都有一個顏色，相鄰可能同色。長度為 m 的連續區段且各種顏色都各出現一次，則稱為「完美彩帶」。請找出總共有多少段可能的完美彩帶。請注意，兩段完美彩帶之間可能重疊。

Time limit: 1 秒

輸入格式：第一行為整數 m 和 n ，滿足 $2 \leq m \leq n \leq 2 \times 10^5$ ；第二行有 n 個以空白間隔的數字，依序代表彩帶從左到右每一格的顏色編號，顏色編號是不超過 10^9 的非負整數，每一筆測試資料的顏色數量必定恰好為 m 。

輸出：有多少段完美彩帶。

範例輸入：

```

4 10
1 4 1 7 6 4 4 6 1 7

```

範例結果：

```

3

```

說明：區間 $[2, 5]$ 是一段完美彩帶，因為顏色 4、1、7、6 剛好各出現一次，此外，區間 $[3, 6]$ 與 $[7, 10]$ 也都是完美彩帶，所以總共有三段可能的完美彩帶。

5.3. 補充說明

正如我們在本章開頭所說的，很多分治演算法所解的問題都有其他版本的解法，本章提出的例題與習題其實都存在別的解法，甚至別的解法的複雜度還更好。其實，從基本的道理上看，很多分治的做法可以轉換成掃描線演算法並不意外。分治法的重點在合併，也就是跨過左邊與右邊的解；而掃描線演算法是一點一點的逐步加入右邊的資料。

如果仔細去看合併排序法與插入排序法，我們可以這麼說：其實合併排序法就是一次處理一整批的插入排序法。但分治的好處也在這裡，因為是整批的合併，所以每個成員參與合併的次數不會太多(典型是 $\log(n)$ 次)，合併時即使不使用什麼高超的技巧或複雜的資料結構，最後還是可以得到不錯的整體複雜度。反觀掃描線演算法，因為一次加入一個資料，所以前面處理的某些結構或部分結果不可以丟棄，因此通常需要有個資料結構來幫忙維護，尤有甚者，這個資料結構往往必須是動態的，因為結構會不斷的變動。也就是說，對於一些題目，如果用分治法，搭配簡單的陣列可能就可以做，但是如果用掃描線演算法就必須配備 STL 中的資料結構，甚至是 STL 中沒有的資料結構。

具體舉例來說，對於 Q-5-5(closest pair)，如果以掃描線算法來作，必須保持已經掃過的點以 Y 值排序，我們必須使用一個動態資料結構(如 STL 中的 `multimap`)，但是分治法就不需要了。以考試或比賽範圍來說，`multimap` 可能不在考試範圍，但是使用分治法並不需要，所以還是可以考。另外一個例子更“有趣”，那是 P-5-4 的反序數量，我們在這一章中已經看到了，將分治法的合併演算法稍加修改就可以解，如果用掃描線演算法，每次碰到 `a[i]`，必須找到前面有幾個數字大於 `a[i]`，很不幸的，STL 中並沒有一個資料結構可以很簡單做到這事情，所以很多程式競賽的選手幾乎都是用線段樹(segment tree)來做這件事。也就是說，如果用分治計算反序數只需要簡單的程式與陣列，但是用掃描線算法則必須裝備一個“頗為複雜”的資料結構(線段樹)。結論是：分治是一個很重要的演算法概念與思維模式。

最後藉著線段樹這個話題，附帶提一下課本上念資料結構與程式競賽的資料結構不一樣的地方。STL 中的 `set` 基本上是個平衡的 BST(binary search tree)，如果這棵 BST 是自己種的，想要在 BST 上查詢比某個數字小的有多少，那只要簡單的修改一下 BST 的結構就可以簡單的做到，但是 STL 雖然有 `BST(set)`，但是沒辦法做這件事，所以練競賽的選手需要學線段樹，當然線段樹的用途很廣同。其實線段樹也就是一種可以偷懶的 BST，因為在競賽的場合，不太可能有時間去自己寫 BST，所以可以用 STL 中的就用，不能用的就會找出替代手法(或是說偷懶手法)，這樣的狀況在競賽程式中屢見不鮮，另外一個很有名的手法就是在優先佇列中降 key 的處理(decreasing key in a priority queue)，不過這些都超過本教材的範圍，僅此做一些額外的說明給有興趣的人參考。